

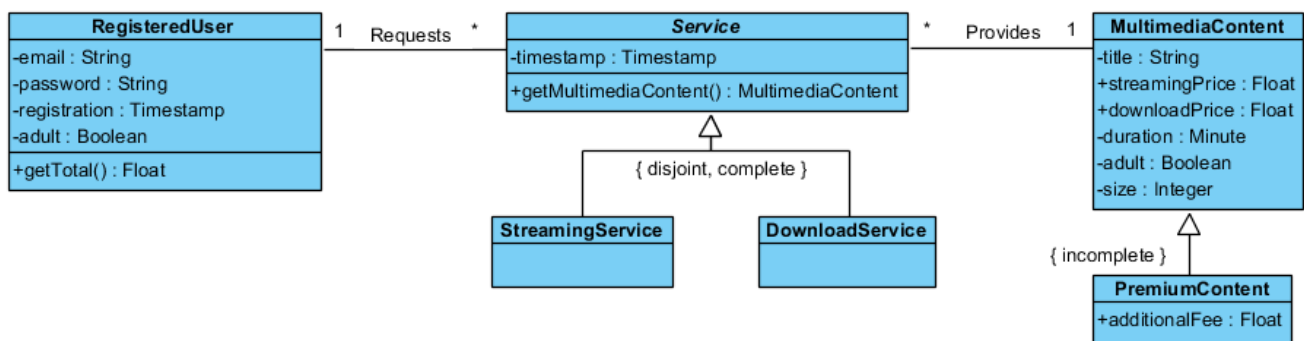
Análisis y Diseño con Patrones

Práctica 1: Principios de diseño, patrones de análisis y arquitectónicos

Nos han contratado para desarrollar un software para gestionar el streaming de películas online. El software formará parte de un sistema mayor, pero sólo tendrá que gestionar información sobre las películas ofrecidas, los suscriptores de la plataforma y las reproducciones y descargas que realizan en la misma.

Obviaremos muchos elementos que un sistema real tendría. La razón de ello no es otra que la de hacer una práctica didáctica, y que la dificultad y dedicación prevista sean las deseadas.

Disponemos ya de un análisis previo que podemos utilizar como punto de partida, pero que habrá que corregir y mejorar:



Claves de las clases:

- *RegisteredUser*: email
- *Service*: timestamp
- *MultimediaContent*: title

Restricciones de integridad:

- Para todo servicio solicitado por un usuario registrado, la fecha y hora del servicio debe ser posterior a la fecha y hora de registro del usuario en la plataforma.
- Contenido multimedia para adultos no puede ser ofrecido a usuarios registrados no adultos.

Como podemos ver en el diagrama de partida, los usuarios tienen una operación que devuelve el importe total pagado por todos los servicios que han solicitado. El precio de un servicio se calcula de la siguiente forma:

- Para todos los usuarios que quieren ver un contenido multimedia por streaming, se aplica el precio de streaming de dicho contenido multimedia.
- Para todos los usuarios que quieren descargar un contenido multimedia de la plataforma, se aplica el precio de descarga de dicho contenido multimedia.
- Si el contenido es premium, en cualquiera de los casos anteriores se añade el cargo adicional especificado en el atributo *additionalFee*.

Supongamos que la implementación de este método es como el siguiente pseudocódigo:

```
public class RegisteredUser
{
    private List<Service> services;

    public float getTotal()
    {
        float total = 0.0;
        foreach (Service s in services)
        {
            MultimediaContent mc = s.getMultimediaContent();
            if(typeof(s)==StreamingService) total += mc.streamingPrice;
            else if (typeof(s)==DownloadService) total += mc.downloadPrice;
            if (typeof(mc)==PremiumContent) total += mc.additionalFee;
        }
        return total;
    }
}
```

Ejercicio 1 (10%)

Revisamos el pseudocódigo de la operación *getTotal* de la clase *RegisteredUser* y nos preocupa que su diseño sea un poco frágil ya que no vemos claro si contempla los posibles escenarios futuros y su impacto:

- cambiamos el tipo de datos de importes o la forma de calcular un cargo adicional a nuestros contenidos premium; y/o
 - ampliamos el sistema para admitir otro tipo de servicio (como la compra en soporte físico).
- a) Indica si este diseño satisface o no la Ley de Demeter y razona tu respuesta.
b) Indica si este diseño satisface o no el principio de diseño Abierto-Cerrado y razona tu respuesta.

Solución:

- a) No, esta solución no satisface la Ley de Demeter, ya que la clase *RegisteredUser*, en el método *getTotal*, utiliza la clase *MultimediaContent*, que no tiene directamente asociada. Es decir, obtiene instancias de *MultimediaContent* que no tiene asociadas, invoca métodos de las mismas y, por tanto, *habla con desconocidos*. En consecuencia, cualquier cambio en esta lógica propia de los contenidos multimedia, afectaría al código de *RegisteredUser*.
- b) No, esta solución viola el principio de diseño Abierto-Cerrado, ya que la clase *RegisteredUser* es consciente de las subclases de *Service* e invoca un método u otro según la subclase a la que pertenece el objeto de cada iteración. Si, más adelante, ampliamos el sistema para admitir una nueva subclase de *Service* deberíamos modificar el código de *RegisteredUser* para obtener el precio correspondiente dentro de un nuevo *else-if*.

Ejercicio 2 (30%)

Propón una solución alternativa (también en pseudocódigo del mismo estilo) que corrija los problemas de la operación *getTotal* de *RegisteredUser* que has detectado en el ejercicio anterior y que por tanto cumpla cualquier principio violado. Realiza todos los cambios que consideres necesarios en cualquiera de las clases del modelo del enunciado.

Solución:

```
public class RegisteredUser
{
    private List<Service> services;

    public float getTotal()
    {
        float total = 0.0;
        foreach (Service s in services) total += s.getServiceFee();
        // esta clase ya sólo “habla” con Service,
        // quien sí se podrá comunicar con MultimediaContent
        // al estar asociada con ella
        return total;
    }
}

public abstract class Service
{
    protected MultimediaContent content;

    // operación plantilla
    public float getServiceFee()
    {
        return this.getSpecificPrice() + content.getAdditionalCharges();
    }

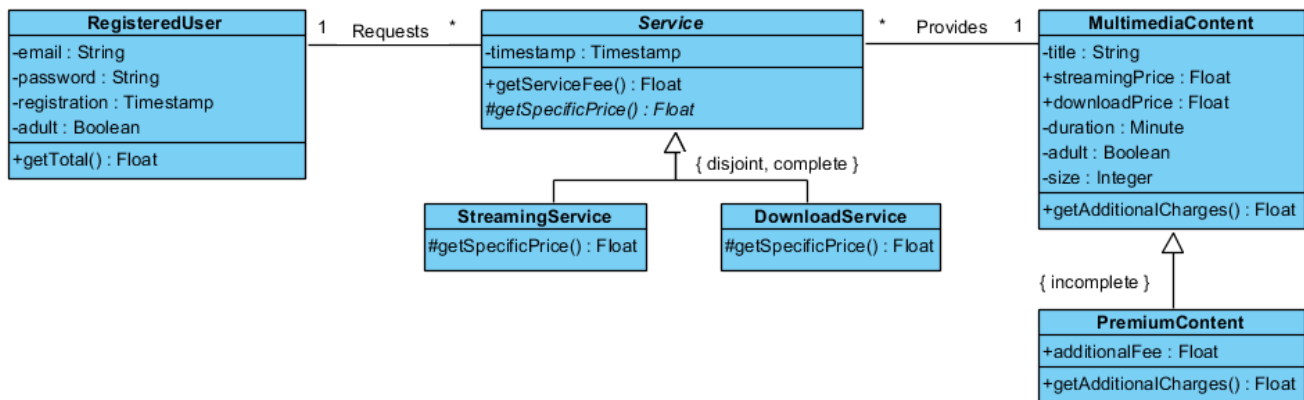
    protected abstract float getSpecificPrice();
    // operación abstracta que será implementada en cada subclase;
    // la decisión de qué precio aplicar depende de la subclase
}
```

```
public class StreamingService inherits Service
{
    protected override float getSpecificPrice()
    {
        return content.streamingPrice;
    }
}

public class DownloadService inherits Service
{
    protected override float getSpecificPrice()
    {
        return content.downloadPrice;
    }
}

public class MultimediaContent
{
    public float getAdditionalCharges()
    {
        return 0.0;
        // operación polimórfica;
        // por defecto, un contenido no tiene cargo adicional
    }
}

public class PremiumContent inherits MultimediaContent
{
    private float additionalFee;
    public overrides float getAdditionalCharges()
    {
        return additionalFee;
        // sólo los premiums tienen cargo adicional, así que
        // sobreescribimos la operación de la superclase para así informarlo
    }
}
```



Ejercicio 3 (30%)

Siguiendo con los precios, nos damos cuenta de que si un contenido cambia de precio (por ejemplo, de streaming), necesitamos que los servicios ya realizados antes de ese cambio no cambien sus precios.

Nos proponemos buscar una solución que nos permita conocer el precio de cada contenido en el momento en que se hizo la solicitud de servicio así como en cualquier momento pasado.

Quisiéramos, pues, que en lugar de un sencillo atributo para cada precio (streaming y descarga), de cada contenido multimedia pudiéramos saber los precios en cualquier momento del pasado y suponed que para ello disponemos de una nueva clase *Price* para representar precios y que ofrece un método *getValue* para obtener el valor de tipo float.

- Describe brevemente cómo solucionarías este problema. En caso de que propongás la aplicación de un patrón, explica cuál y razona su conveniencia.
- Propón una solución detallada en forma de diagrama de clases. Muestra sólo las clases que cambien respecto al diagrama del enunciado.
- Indica cómo sería el pseudocódigo resultante para que un contenido multimedia devuelva el último precio de streaming que está en vigor.

Solución:

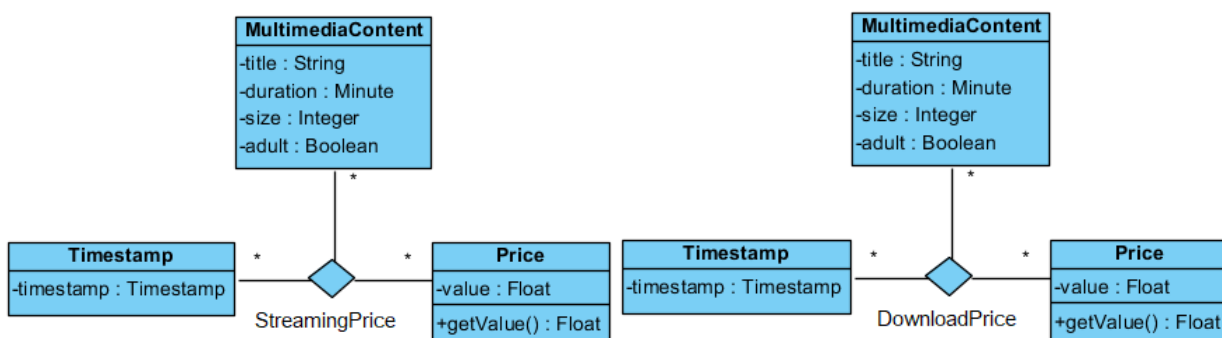
a)

Ahora mismo el precio de un contenido multimedia es un atributo del mismo y, por tanto, de cada contenido sólo sabemos el precio actual. A pesar de que se trata de un atributo y no de una asociación, el patrón que nos permite resolver esta problemática es el patrón Asociación Histórica.

Este patrón nos permite transformar una asociación (o un atributo, como es el caso) en un histórico de los valores que ha tomado la asociación o atributo a lo largo del tiempo. Este es exactamente el problema que queremos resolver.

b)

Como lo que tenemos es un atributo, necesitamos aplicar la variación en que primero extraemos el atributo como una asociación. El resultado final es:

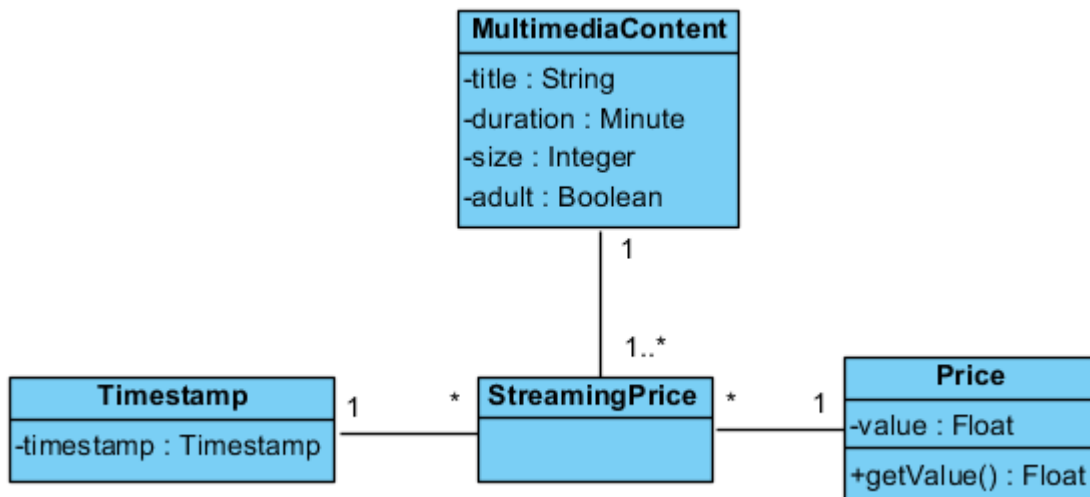


Este diagrama refleja una restricción que antes no teníamos: dado un contenido y una fecha-hora (instante o timestamp) concreta, sólo podemos tener un precio (para cada tipo de servicio). Es decir, un contenido sólo puede tener un cambio de precio en un instante dado.

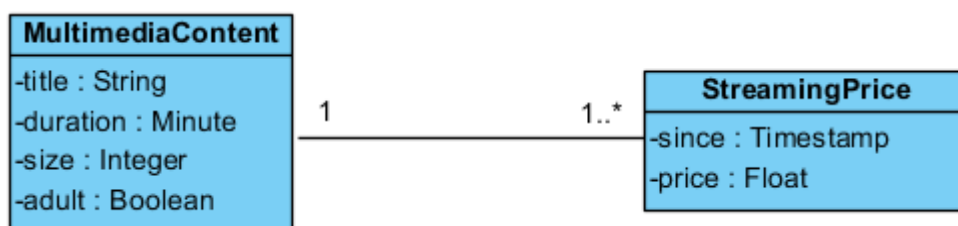
En esta solución asumimos que el precio de un contenido es el actual hasta que aparece una asociación con un *Timestamp* posterior, momento en que aquel precio pasa a ser histórico y ya no es el actual. Por lo tanto:

- Estamos ya reflejando que un contenido multimedia no puede tener dos precios del mismo servicio al mismo tiempo, ya que siempre asumimos que si hay dos asociaciones cada una representa un precio que deja de ser válido cuando comienza el siguiente.
- Estamos asumiendo que un contenido NO puede tener períodos de tiempo en que no tiene ningún precio para cada servicio.

Por otra parte, con los lenguajes orientados a objetos actuales no se suele permitir utilizar asociaciones ternarias. Se puede hacer modelando la asociación histórica como si tuviera una clase asociativa y luego haciendo una pequeña transformación. Por ejemplo, con *StreamingPrice*:

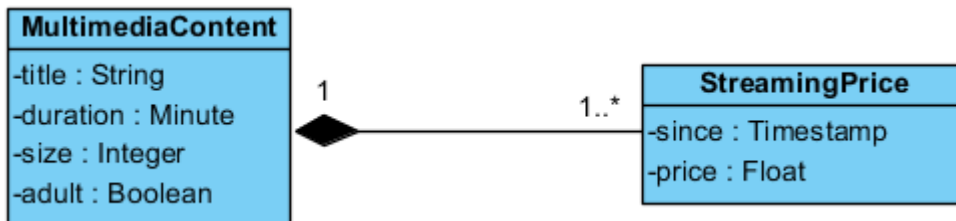


De hecho, como *Timestamp* y *Price* los podríamos considerar tipos de datos, que normalmente pondríamos como atributos, esta solución la podemos transformar en:



Por último, podemos ver que tanto *StreamingPrice* como *DownloadPrice* son componentes de un contenido multimedia, ya que pertenecen a un y sólo un contenido multimedia, no tiene sentido que

lo cambiamos de contenido y si destruimos un contenido tendremos que destruir su histórico de precios. Por lo tanto, podemos modelarlo como:



c)

```
public class MultimediaContent
{
    private List<StreamingPrice> streamingHistory;

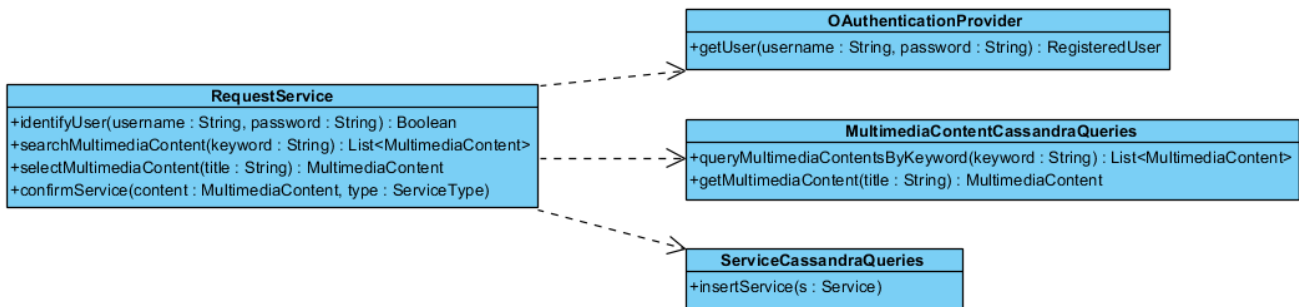
    public float getCurrentStreamingPrice()
    {
        if (streamingHistory.isEmpty()) return null;
        return streamingHistory.last().price;
    }
}

public class StreamingPrice
{
    public Timestamp since;
    public float price;
}
```

Ejercicio 4 (20%)

Empezamos ahora a diseñar la arquitectura del software del sistema y hemos decidido utilizar una arquitectura con 3 capas: presentación, dominio y servicios técnicos.

Durante el diseño de los primeros elementos nos encontramos en un punto en que disponemos de una clase *RequestService* de la capa de dominio que nos permite ir dando respuesta, a nivel de abstracción de la lógica, al caso de uso por el que un usuario registrado se autentica, busca entre el contenido multimedia disponible, selecciona uno de su gusto y solicita reproducirlo por streaming o realizar una descarga.



A medida que se van realizando las diferentes interacciones del caso de uso, se van ejecutando los métodos de *RequestService*:

- *identifyUser*: Invocada por un subsistema de seguridad que informa cuál es el usuario que está utilizando el sistema. La clase *RequestService* puede buscar los datos del usuario utilizando *OAuthenticationProvider*. Si es efectivamente obtenido desde *OAuthenticationProvider*, el caso de uso puede continuar.
- *searchMultimediaContent*: Obtiene un listado de contenidos multimedia disponibles en la plataforma que cumplen los criterios de búsqueda en el título. La clase *RequestService* puede buscar los datos de los contenidos multimedia utilizando *MultimediaContentCassandraQueries*.
- *selectMultimediaContent*: Obtiene un contenido multimedia a partir de la selección de título que realiza el usuario sobre el listado obtenido en la operación anterior. La obtención del objeto concreto se realiza utilizando *MultimediaContentCassandraQueries*.
- *confirmService*: Crea el servicio a partir del contenido multimedia escogido y el tipo de servicio deseado (streaming o descarga), y lo registra en la base de datos mediante *ServiceCassandraQueries*. También, obtiene los binarios del contenido multimedia en el formato óptimo para el reproductor de nuestra plataforma, si ha seleccionado streaming, o bien obtiene el fichero descargable, si ha seleccionado descarga; pero no representamos esto último en aras de simplificar el modelo.

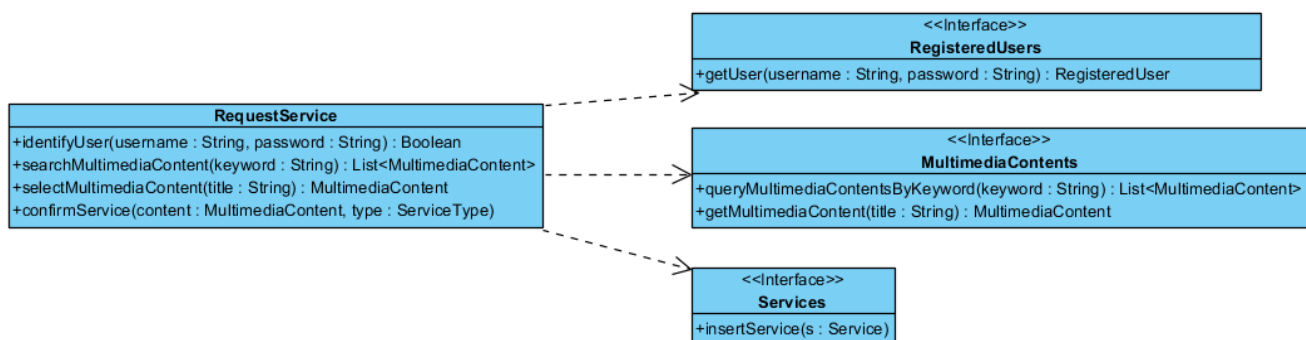
Las clases *OAuthenticationProvider*, *MultimediaContentCassandraQueries* y *ServiceCassandraQueries*, como podéis deducir, son de la capa de servicios técnicos.

¿Crees que esta clase cumple todos los principios de diseño que hemos visto? En caso afirmativo, justifica por qué crees que es así. En caso negativo, indica qué principios crees que no cumple e indica qué cambios introducirías en el diagrama de clases para cumplirlos.

Solución:

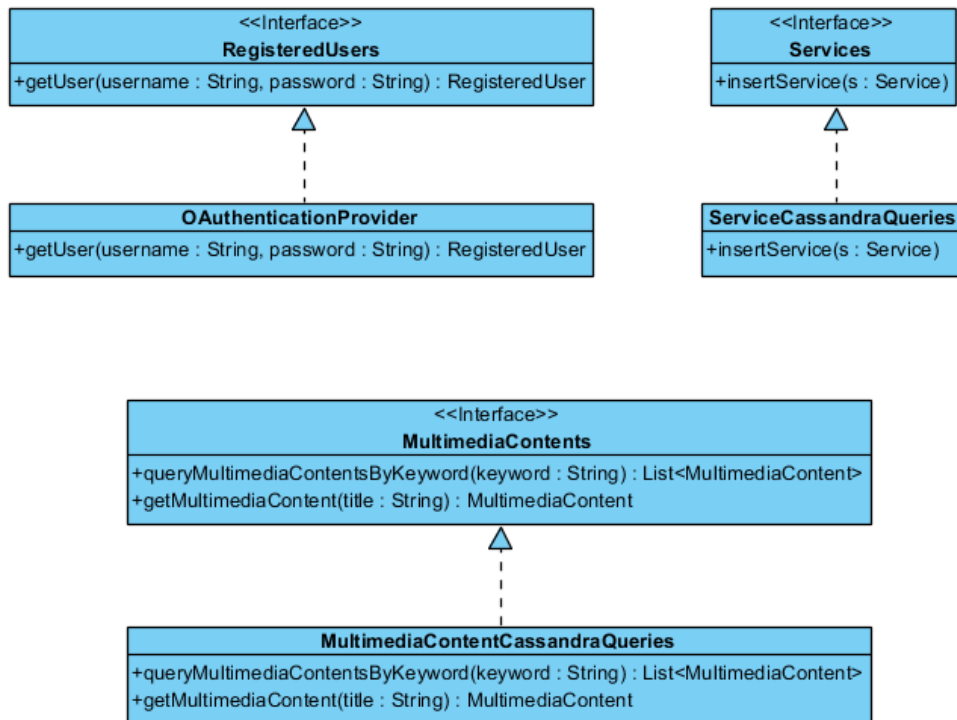
Esta solución se puede cuestionar si satisface más o menos algunos de los principios de diseño. Pero un principio que incumple claramente es el principio de Inversión de Dependencias, ya que *RequestService*, que está al nivel de abstracción de la lógica del dominio, depende de clases que tienen un nivel de abstracción mucho más bajo, ya que tienen el nivel de abstracción de la infraestructura; de hecho, el de un fabricante concreto de bases de datos no relacionales y el de un método popular de autenticación.

A nivel de diagrama de clases la solución consiste en hacer que *RequestService* dependa de abstracciones y que los detalles también dependan de estas abstracciones. Una forma de lograrlo es crear una abstracción para cada clase demasiado concreta:



En este caso hemos creado 3 interfaces que representan, al nivel de abstracción del dominio, el conjunto de usuarios, contenidos multimedia y servicios solicitados que tenemos en el sistema. Ahora *RequestService* no depende de detalles sino de otra abstracción que está a su mismo nivel de abstracción.

Entonces, los detalles, que ya teníamos, pasan a depender de las abstracciones. Una forma de dependencia válida es la de implementación de interfaz:



Ejercicio 5 (10%)

En el ejercicio anterior hemos planteado una cuestión de diseño.

Veamos un fragmento del pseudocódigo de *RequestService*:

```
class RequestService
{
    ...
    public MultimediaContent selectMultimediaContent (string title)
    {
        // ups! no sabemos de dónde obtener la instancia de
        // MultimediaContentCassandraQueries que necesitamos:
        MultimediaContentCassandraQueries queries = ???;
        MultimediaContent mc = queries.getMultimediaContent(title);
        return mc;
    }
    ...
}
```

Visto el snippet de código anterior, no sabemos de dónde obtener una instancia de *MultimediaContentCassandraQueries*. Podríamos convertir todas las operaciones de esta clase en operaciones con ámbito de clase (estáticas) y usarlas directamente desde *RequestService*. Pero nos preocupa que quizás más adelante querramos soportar también la base de datos *ElastiCache* (y poder decidir en cada instalación si utilizamos una u otra base de datos); en tal caso tendríamos *RequestService* acoplado a la clase *MultimediaContentCassandraQueries* y sería complicado poder decidir la base de datos en el momento de la configuración del software.

Propón una solución que permita desacoplar *RequestService* de las clases específicas de Cassandra. Esta solución debe estar en consonancia con la solución que has propuesto para el ejercicio anterior. La solución debe explicar, además, cómo *RequestService* consigue la instancia

de la clase que utilizará para consultar un contenido multimedia a partir de su nombre, resolviendo así la duda surgida en el pseudocódigo del ejercicio anterior.

Solución:

En la solución del ejercicio anterior, ya hemos modelado que *RequestService* solo use (invoque) métodos de abstracciones, interfaces pertenecientes a la clase de dominio, como *MultimediaContents*, *RegisteredUsers*, y *Services*.

En algún momento, sin embargo, necesitamos obtener una instancia de este tipo. Si *RequestService* instanciara ella misma las instancias que necesita se volvería a acoplar a las clases concretas de la capa de servicios técnicos:

```
MultimediaContents contents = new MultimediaContentCassandraQueries();
```

Queremos, pues, escoger la implementación de *MultimediaContents* (y demás) a utilizar sin acoplar *RequestService* a esa implementación. Este es exactamente el problema que resuelve el patrón de arquitectura Inyección de Dependencias.

La solución consiste en asociar una instancia de *MultimediaContents*, una de *RegisteredUsers* y una de *Services* a *RequestService*. Bajando al nivel de código, *RequestService* tendrá un atributo para cada una de estas instancias. Entonces, en lugar de inicializar la misma *RequestService* estos atributos, le inyectaremos dichas dependencias desde fuera. Proponemos utilizar la variante en que se inyectan a través del constructor:

```
class RequestService
{
    private MultimediaContents contents;
    private RegisteredUsers users;
    private Services services;

    public RequestService (
        MultimediaContents cs,
        RegisteredUsers us,
        Services ss)
    {
        this.contents = cs;
    }
}
```

```
        this.users = us;
        this.services = ss;
    }

    public void identifyUser (string email)
    {
        RegisteredUser ru = users.getRegisteredUser(email);
        ...
    }
}
```

Hemos mostrado el diseño resultante y cómo se utilizaría una de las abstracciones desde una de las operaciones de la clase.